

[학기]Base-W1] GPT & DeepSeek

🕒 생성일	2026년 9월 2일 오후 4:03
🏷️ 태그	

▼ [2018][arXiv][GPT]

1. Abstract & Introduction

라벨 없는 대량의 텍스트로 **generative pre-training**을 하고, 태스크마다 **입력 형태만 바꿔 fine-tuning**하면 task별 전용 아키텍처 없이도 12개 중 9개 태스크에서 SOTA를 찍을 수 있다.

1-1. Problems

- NLP 대부분의 태스크는 **labeled data가 희소**한 반면, unlabeled text는 사실상 무한
- word embedding(word2vec, GloVe)은 word-level 정보만 전이 → 문장·문맥 수준 표현은 매번 처음부터 학습
- 태스크마다 **전용 아키텍처**를 새로 설계해야 해서 확장성이 떨어짐

해결해야 할 두 가지:

1. 어떤 **objective**가 전이에 유리한 표현을 만드는가 (LM / MT / auto-encoding ...)
2. 학습된 표현을 target task로 **어떻게 옮길 것인가**

1-2. Core Idea

- 2단계 **semi-supervised** : **unsupervised pre-training** → **supervised fine-tuning**
- **Architecture** : Transformer **Decoder** (LSTM 대비 long-range dependency에 강함 → 전이 성능이 안정적)
- **Transfer 방식** : 구조를 바꾸지 않고 **input transformation**(traversal-style)으로 입력만 하나의 연속 시퀀스로 변환

⇒ **task-agnostic한 단일 모델**로 분류·함의·유사도·QA를 모두 처리

1-3. Contribution

1. 최소한의 구조 변경만으로 **12개 중 9개 태스크 SOTA** (Section 3)
2. **Transformer가 전이학습에 유리한 구조임**을 LSTM과의 비교로 실증
3. pre-training만으로 태스크 능력이 생기는 **zero-shot 행동**을 분석

2. Method

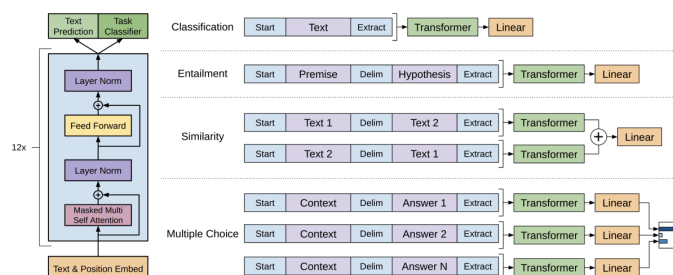


Figure 1: (left) Transformer architecture and training objectives used in this work. (right) Input transformations for fine-tuning on different tasks. We convert all structured inputs into token sequences to be processed by our pre-trained model, followed by a linear+softmax layer.

2-1. Stage 1. Unsupervised Pre-training

앞의 k 개 토큰으로 다음 토큰을 맞추는 **평범한 language modeling**.

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

$$h_0 = UW_e + W_p, \quad h_l = \text{transformer_block}(h_{l-1}), \quad P(u) = \text{softmax}(h_n W_e^\top)$$

▼ [Training Settings] GPT-1

- **12-layer decoder-only Transformer**, hidden 768, head 12, FFN 3072, GELU
- position embedding은 sinusoidal이 아니라 **learned**
- BPE vocabulary (40,000 merges), context 512, dropout 0.1
- Adam, max lr 2.5e-4 + cosine schedule, batch 64, **100 epochs**
- 코퍼스: **BooksCorpus (7,000권)**
 - ⇒ 문장 단위로 셔플된 1B Word Benchmark와 달리 **긴 연속 문단**이 그대로 남아 있어 long-range dependency 학습에 유리

2-2. Stage 2. Supervised Fine-tuning

pre-trained 마지막 블록의 출력 h_l^m 에 **linear layer 하나만** 얹어 라벨을 예측.

$$P(y | x^1, \dots, x^m) = \text{softmax}(h_l^m W_y), \quad L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \dots, x^m)$$

Auxiliary objective : fine-tuning 중에도 LM loss를 같이 엮음

$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda L_1(\mathcal{C})$$

- 효과: **generalization 향상 + 수렴 가속**
- 단, 큰 데이터셋에서만 도움이 되고 작은 데이터셋에서는 이득이 없음 (3-2 ablation)
- 새로 학습되는 파라미터는 W_y 와 delimiter embedding **뿐**

2-3. Task-specific Input Transformation

구조를 건드리지 않고, 입력을 하나의 연속 시퀀스로 변환해서 넣는 것이 핵심 트릭.

- **Classification** : `[start] text [extract]`
- **Entailment** : premise + `$` delimiter + hypothesis
- **Similarity** : 두 문장의 순서를 바꾼 두 시퀀스를 각각 통과시켜 element-wise 합
 - ⇒ 유사도에는 고유한 순서 관계가 없기 때문
- **Multiple choice** : (context + `$` + 각 answer)를 후보 수만큼 각각 통과 → 전체를 softmax

3. Experiment

3-1. 주요 결과

총 12개 태스크 중 9개에서 SOTA, GLUE 종합 점수 **68.9 → 72.8**.

Table 2: Experimental results on natural language inference tasks, comparing our model with current state-of-the-art methods. 5x indicates an ensemble of 5 models. All datasets use accuracy as the evaluation metric.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>	-	-
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

Table 3: Results on question answering and commonsense reasoning, comparing our model with current state-of-the-art methods.. 9x means an ensemble of 9 models.

Method	Story Cloze	RACE-m	RACE-h	RACE
val-LS-skip [55]	76.5	-	-	-
Hidden Coherence Model [7]	<u>77.6</u>	-	-	-
Dynamic Fusion Net [67] (9x)	-	55.6	49.4	51.2
BiAttention MRU [59] (9x)	-	<u>60.2</u>	<u>50.3</u>	<u>53.3</u>
Finetuned Transformer LM (ours)	86.5	62.9	57.4	59.0

Table 4: Semantic similarity and classification results, comparing our model with current state-of-the-art methods. All task evaluations in this table were done using the GLUE benchmark. (*mc*= Mathews correlation, *acc*=Accuracy, *pc*=Pearson correlation)

Method	Classification		Semantic Similarity			GLUE
	CoLA (mc)	SST2 (acc)	MRPC (F1)	STS (pc)	QQP (F1)	
Sparse byte mLSTM [16]	-	93.2	-	-	-	-
TF-KLD [23]	-	-	86.0	-	-	-
ECNU (mixed ensemble) [60]	-	-	-	<u>81.0</u>	-	-
Single-task BiLSTM + ELMo + Attn [64]	35.0	90.2	80.2	55.5	66.1	64.8
Multi-task BiLSTM + ELMo + Attn [64]	18.9	91.6	83.5	72.8	63.3	<u>68.9</u>
Finetuned Transformer LM (ours)	45.4	91.3	82.3	82.0	70.3	72.8

- 특히 긴 문맥이 필요한 태스크(Story Cloze, RACE)에서 격차가 큼 ⇒ Transformer의 long-range 처리력이 그대로 반영
- 데이터셋 크기와 무관하게(5.7k ~ 550k) 고르게 작동

3-2. Analysis

(1) 전이하는 layer 수의 영향

- pre-trained layer를 많이 옮길수록 성능이 단조 증가, MultiNLI 기준 최대 9% 향상
- ⇒ 각 층이 target task에 쓸모 있는 기능을 따로따로 담고 있다는 의미

(2) Zero-shot Behaviors

- fine-tuning 없이, pre-training update가 늘어날수록 태스크 성능이 꾸준히 상승
- ⇒ LM 학습 자체가 다양한 태스크에 필요한 능력을 부수적으로 습득하고 있음
- 같은 실험에서 LSTM은 점수도 낮고 **variance가 훨씬 큼**

(3) Ablation

- auxiliary LM objective 제거 → 큰 데이터셋(NLI, QQP)에서만 성능 하락
- Transformer → LSTM 교체 → 평균 5.6점 하락
- pre-training 없이 바로 지도학습 → 평균 14.8% 하락 (전 태스크에서 하락)

4. Conclusion & Limitation

태스크마다 모델을 새로 설계하는 대신, 하나의 generative pre-trained 모델 + 입력 변환만으로 광범위한 NLU 태스크를 커버할 수 있다.

- 비지도 pre-training이 어떤 태스크에서도 손해가 아니며, 특히 데이터가 적을수록 이득이 큼

- 이후 GPT-2 / GPT-3로 이어지는 **scaling + zero-shot** 노선의 출발점

4-1. Limitations

- **Unidirectional** : left-to-right LM이라 양방향 문맥을 못 씀 → 다음 해 **BERT**가 정확히 이 지점을 공략
- 여전히 **fine-tuning 필요** : 태스크마다 labeled data와 별도 학습이 요구됨 → GPT-2/3의 zero-few-shot으로 이어지는 문제 의식
- **입력 변환의 수작업성** : 태스크마다 delimiter-순서를 사람이 설계해야 함

▼ [2025][Nature][DeepSeek-R1]

1. Abstract & Introduction

사람이 만든 reasoning trace 없이, **정답 여부만 보는 순수 RL**만으로 self-reflection-verification 같은 고차원 추론 행동을 **발현 (emerge)**시킬 수 있다.

1-1. Problems

기존의 CoT prompting / post-training SFT 방식의 한계:

- **human-annotated reasoning trace에 의존** → scalability 저하 + 인간의 cognitive bias 유입
- 모델이 인간의 사고 과정을 **복제**하도록 강제됨 → 성능 상한이 **인간 예시에 갇힘**
- ⇒ 인간보다 나은 **non-human-like reasoning pathway**의 탐색이 원천 차단

⇒ **SFT 단계를 아예 건너뛰고**, base model 위에 곧바로 RL을 걸어 self-evolution을 관찰하자.

1-2. Core Idea

- **Base model** : DeepSeek-V3-Base (MoE, total 671B / activated 37B)
- **RL algorithm** : GRPO (Group Relative Policy Optimization)
- **Reward** : 최종 예측이 ground-truth와 맞는지 그것만. 추론 과정 자체에는 제약 없음
- **가설** : human-defined reasoning pattern이 exploration을 제한한다. 제약 없는 RL이 오히려 새로운 추론 능력의 emergence를 유도한다.

결과물 → **DeepSeek-R1-Zero**

가르친 적이 없는데도 응답이 점점 길어지고, verification / reflection / alternative approach 탐색이 자발적으로 등장.

1-3. Contribution

1. **DeepSeek-R1-Zero** : SFT 전혀 없이 pure RL만으로 reasoning 능력 발현
2. **DeepSeek-R1** : R1-Zero의 3대 약점(가독성 / language mixing / 좁은 도메인)을 multi-stage pipeline으로 해결
3. **Distillation** : 대형 모델의 emergent reasoning pattern을 소형 모델로 전이

2. DeepSeek-R1-Zero

SFT 없이, DeepSeek-V3-Base에 **GRPO + rule-based reward**만 걸어서 학습.

2-1. GRPO

PPO의 **value model(critic)**을 제거하고, 같은 질문에 대해 뽑은 G 개 응답 **그룹 내부의 상대적 점수**로 advantage를 추정.

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E} \left[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q) \right] \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) \right) \right).$$

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

왜 value model을 버렸나 (vs PPO)

- value model은 보통 policy와 비슷한 크기라 메모리·연산 오버헤드가 큼
- outcome reward만 있는 상황에서 **partial response**로 최종 reward를 예측하는 것 자체가 어려움
- 특히 long CoT에서는 앞 내용이 뒤에서 reflection으로 뒤집히므로 사실상 불가능

2-2. Reward Design

수학·코드·논리 도메인에 **rule-based reward**만 사용.

$$Reward_{rule} = Reward_{acc} + Reward_{format}$$

- **Accuracy reward** : 수학은 지정된 형식의 최종 답을 규칙으로 검증, 코드는 **컴파일러 + test case**로 판정
- **Format reward** : 사고 과정을 `<think> ... </think>` 안에 담도록 강제

neural reward model을 쓰지 않은 이유 : 대규모 RL에서 **reward hacking**에 취약하고, RM 재학습 비용과 파이프라인 복잡도가 크기 때문.

2-3. Incentivize Reasoning Capability

(1) 성능 (AIME 2024)

- pass@1 **15.6% → 77.9%**, self-consistency(cons@16) 적용 시 **86.7%**

(2) Self-evolution

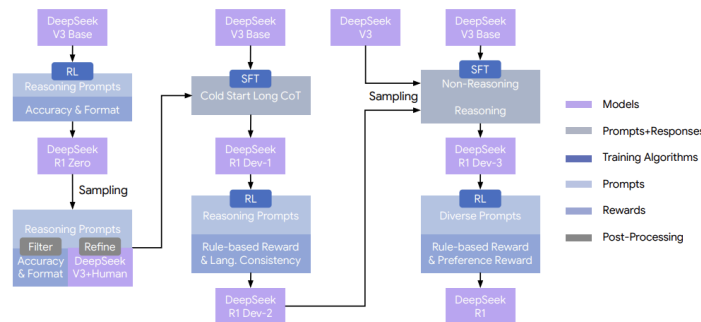
- 학습이 진행될수록 **thinking time(응답 길이)**이 꾸준히 증가
- 외부 개입이 아니라 순전히 **intrinsic adaptation**의 결과

(3) "Aha Moment"

- 특정 시점에 **"wait"**의 빈도가 급격히 **spike** → 스스로 이전 단계를 재평가하기 시작
- reflective word("wait", "mistake", "verify" 등) 빈도가 학습 초기 대비 **5~7배** 증가

⇒ 어떻게 풀지를 가르친 게 아니라, 올바른 인센티브만 주었더니 스스로 문제해결 전략을 개발했다.

3. DeepSeek-R1



R1-Zero의 약점(가독성 / language mixing / 좁은 도메인)을 **4단계 multi-stage pipeline**으로 해결.

$$V3\text{-Base} \xrightarrow{\text{Cold Start SFT}} \text{Dev1} \xrightarrow{\text{1st RL}} \text{Dev2} \xrightarrow{\text{Rejection Sampling + SFT}} \text{Dev3} \xrightarrow{\text{2nd RL}} \text{R1}$$

1. **Cold Start** : 대화체·human-aligned long CoT 데이터 수천 건으로 SFT
2. **1st RL** : 추론 능력 + **language consistency** 개선
3. **Rejection Sampling + SFT** : non-reasoning 데이터까지 함께 넣어 writing 능력 확보
4. **2nd RL** : helpfulness · harmlessness 정렬

3-1. Model-based Rewards

general data는 규칙으로 채점이 불가능 → **reward model**로 human preference를 포착.

- **Helpfulness** : 최종 summary만 평가 → reasoning 과정에 대한 간접 최소화
- **Harmlessness** : reasoning process + summary 전체 평가 → 생성 도중의 risk까지 포착
- **Language consistency** : CoT에서 target language 단어 비율을 그대로 reward에 더함
 - 성능은 미세하게 하락하지만 **가독성이 크게 좋아지는** 의도적 trade-off

model-based preference reward로 오래 학습하면 **reward hacking**이 발생 → 마지막 400 step에만 투입.

4. Experiment

4-1. Stage별 성능 변화 (Table 3)

	Benchmark (Metric)	R1-Zero	R1-Dev1	R1-Dev2	R1-Dev3	R1
English	MMLU (EM)	88.8	89.1	91.2	91.0	90.8
	MMLU-Redux (EM)	85.6	90.0	93.0	93.1	92.9
	MMLU-Pro (EM)	68.9	74.1	83.8	83.1	84.0
	DROP (3-shot F1)	89.1	89.8	91.1	88.7	92.2
	IF-Eval (Prompt Strict)	46.6	71.7	72.0	78.1	83.3
	GPQA Diamond (Pass@1)	75.8	66.1	70.7	71.2	71.5
	SimpleQA (Correct)	30.3	17.8	28.2	24.9	30.1
	FRAMES (Acc.)	82.3	78.5	81.8	81.9	82.5
	AlpacaEval2.0 (LC-wins)	24.7	50.1	55.8	62.1	87.6
	ArenaHard (GPT-4-1106)	53.6	77.0	73.2	75.6	92.3
Code	LiveCodeBench (Pass@1-COT)	50.0	57.5	63.5	64.6	65.9
	Codeforces (Percentile)	80.4	84.5	90.5	92.1	96.3
	Codeforces (Rating)	1444	1534	1687	1746	2029
	SWE Verified (Resolved)	43.2	39.6	44.6	45.6	49.2
	Aider-Polyglot (Acc.)	12.2	6.7	25.6	44.8	53.3
Math	AIME 2024 (Pass@1)	77.9	59.0	74.0	78.1	79.8
	MATH-500 (Pass@1)	95.9	94.2	95.9	95.4	97.3
	CNMO 2024 (Pass@1)	88.1	58.0	73.9	77.3	78.8
Chinese	CLUEWSC (EM)	93.1	92.8	92.6	91.6	92.8
	C-Eval (EM)	92.8	85.7	91.9	86.4	91.8
	C-SimpleQA (Correct)	66.4	58.8	64.2	66.9	63.7

핵심 관찰

- **R1-Zero → Dev1 (Cold Start SFT)** : instruction-following이 크게 개선되지만, 데이터 규모가 작아 **reasoning**은 일부 후퇴 (AIME 77.9 → 59.0)
- **Dev1 → Dev2 (1st RL)** : code / math 등 고난도 추론이 뚜렷하게 향상, 반면 general-purpose 지표는 소폭 개선에 그침
- **Dev2 → Dev3 (Rejection Sampling + SFT)** : non-reasoning 코퍼스 투입 → **AlpacaEval, Aider**가 크게 상승
- **Dev3 → R1 (2nd RL)** : 수학·코드는 이미 확보되어 marginal, 최종 향상은 **instruction-following**과 **user-preference**에 집중

⇒ 각 stage가 담당하는 능력이 명확히 분리되어 나타남.

5. Conclusion & Limitation

pre-trained checkpoint는 이미 복잡한 추론의 **잠재력을 내재**하고 있다. 그것을 여는 열쇠는 대규모 human annotation이 아니라 **어려운 문제 + 신뢰할 수 있는 verifier + 충분한 연산**이다.

5-1. Capability Limitations

- **Tool Use** : 구조화 출력이 약하고 search engine-calculator 같은 도구를 쓸 수 없음
- **Token Efficiency** : 쉬운 질문에 과하게 사고하는 **overthinking**이 관측됨
- **Language Mixing** : 중·영에 최적화되어 제3의 언어로 물어도 영어로 추론·응답

- **Prompting** : prompt에 민감하며 **few-shot**이 일관되게 성능을 떨어뜨림 ⇒ zero-shot 권장

5-2. Pure RL의 근본 한계: Reward Hacking

- pure RL의 성공은 신뢰할 수 있는 **reward 신호**에 전적으로 의존
- writing처럼 규칙화가 어려운 **task**에서는 rule-based RM을 만들 수 없고, 모델이 reward를 주면 policy가 **shortcut**을 찾아 해킹하게 됨
- ⇒ 신뢰 가능한 평가가 없는 task에서 **pure RL**을 **scaling**하는 것은 여전히 **open challenge**

전망 : verifier로 평가 가능한 task라면 난이도와 무관하게 pure RL로 풀릴 여지가 크다. 추론 과정에서 compiler·search engine 같은 외부 도구를 쓰는 **tool-augmented reasoning**이 유망한 방향.